

Figure 1: Multithreaded application design model

registers and stack. The global and static variables present in the process are common to all threads. Each thread will do a specific task assigned by the process.

We considered the boss/worker model as shown in Figure 1 for writing the sample application. Here, the idea is to have a single boss thread (main thread) that creates work and several worker threads that process the work. Typically, the boss thread creates a certain number of workers, distributes the work, and then waits for the workers to complete the work. The worker threads form a thread pool as shown in Figure 1. This model works best when the work items/data are independent of each other and when we can schedule each worker thread on a separate core.

To create a thread, we use the `pthread_creation` function present in the API. We need to call it with four parameters, which are listed below.

Parameter 1: It specifies the address of the variable where we want to store the ID of the newly created thread in the program for future reference.

Parameter 2: It specifies the thread attributes. NULL means default attributes.

Parameter 3: It specifies the address of the function according to which the newly created thread will perform the task.

Parameter 4: It specifies the data which the newly created thread gets from the caller.

Serial (sequential) application performance monitoring

We consider the multiplication of two $N \times N$ matrices as the benchmark application. Matrix multiplication can be done in serial fashion as shown below:

```
// serial program to multiply two nxn
matrices -matrix_mul_serial.c
#include <stdio.h>
#include <stdlib.h>
// global space
```

```
#define n 5120
int a[n][n], b[n][n], c[n][n];

int main()
{
    int i, j, k;
    for(i=0; i<n; i++)          // Data Initialization
    for(j=0; j<n; j++)
    {
        [i][j]=1;
        b[i][j]=1;
    }
    for(i=0; i<n; i++)          // Multiplication
    for(j=0; j<n; j++)
    for(k=0; k<n; k++)
        c[i][j]=c[i][j]+a[i][k]*b[k][j];

    /* printf("\n The resultant matrix is \n");

    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
            printf(" %d", c[i][j]);
        printf("\n");
    }
    */
}
```

The execution is as follows:

```
$ cc -o matrix_ser_5120 matrix_mul_serial.c
$ time ./matrix_ser_5120
```

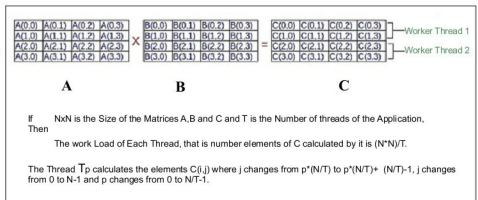


Figure 2: Matrix multiplication in parallel

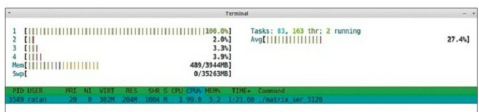


Figure 3: Serial application performance monitoring with htop